Graph RAG 정리

1. Graph RAG란?

Graph RAG는 기존 RAG에 **그래프 구조**, 즉 엔티티와 관계를 추가한 검색·생성 방식이다.

일반 RAG는 보통 다음처럼 동작한다.

```
질문 → 관련 문서 chunk 검색 → LLM에게 context로 제공 → 답변 생성
```

Graph RAG는 여기에 문서 속 개념들의 관계를 추가한다.

```
문서 → 엔티티 추출 → 관계 추출 → 지식 그래프 생성 → 그래프 기반 검색 → 답변 생성
```

즉, 일반 RAG가 "관련 문단"을 찾는 방식이라면, Graph RAG는 "관련 문단 + 그 문단 속 대상들이 어떻게 연결되어 있는지"까지 함께 찾는 방식이다.

---

2. 왜 필요한가?

기존 RAG는 특정 문서나 조항을 찾는 데 강하다.

예를 들어 다음 질문은 일반 RAG로도 잘 처리할 수 있다.

> "계약서의 해지 조항은 뭐야?"

하지만 다음과 같은 질문은 일반 RAG만으로 어렵다.

> "지난 분기 고객 피드백의 핵심 리스크는 뭐야?"
> "A 고객사의 문제와 B 기능 요청은 어떤 관계가 있어?"
> "이 프로젝트가 왜 지연됐는지 관련 회의록과 의사결정을 종합해줘."

이런 질문은 단순히 비슷한 chunk 몇 개를 찾는 것보다, 여러 문서에 흩어진 **사람, 조직, 기능, 사건, 의사결정, 리스크 간 관계**를 파악해야 한다.

Graph RAG는 바로 이 관계 기반 질문에 강하다.

---

3. 핵심 개념

Graph RAG는 문서에서 다음을 추출한다.

엔티티

문서 속 주요 대상이다.

예:

```
ACME 고객사
SSO 기능
Platform Team
Security Review
온보딩 지연
Q3 매출 리스크
```

관계

엔티티 사이의 연결이다.

예:

```
ACME 고객사 --blocked_by--> SSO 기능 지연
SSO 기능 --delayed_by--> Security Review
Platform Team --owns--> SSO 기능
SSO 기능 지연 --risk_to--> Q3 매출
```

그래프

엔티티는 **노드**, 관계는 **엣지**가 된다.

```
(Customer: ACME)
    |
    | blocked_by
    v
(Feature: SSO)
    |
    | delayed_by
    v
(Process: Security Review)
```

---

4. 일반 RAG와 Graph RAG 비교

구분	일반 RAG	Graph RAG
검색 단위	문서 chunk	엔티티, 관계, subgraph, chunk

구분	일반 RAG	Graph RAG
검색 방식	벡터 유사도 중심	그래프 탐색 + 벡터 검색 + 요약
강점	특정 사실 검색	관계, 맥락, 전체 요약
약점	전체 구조 파악이 약함	구축 비용과 복잡도가 높음
적합한 질문	“환불 정책이 뭐야?”	“고객 불만의 주요 원인이 어떻게 연결돼?”

---

5. Graph RAG 동작 흐름

5.1 문서 수집

회사 내부 문서, 회의록, 고객 피드백, 티켓, 위키, 논문, 계약서 등을 모은다.

```
docs/
  customer_feedback.md
  sprint_notes.md
  incident_report.md
  roadmap.md
```

5.2 Chunking

문서를 적당한 크기로 나눈다.

```
문서 A
 ├─ chunk A-1
 ├─ chunk A-2
 └─ chunk A-3
```

이 단계는 일반 RAG와 비슷하다.

5.3 엔티티 추출

문서 chunk에서 중요한 대상을 뽑는다.

예문:

```
ACME 고객사는 SSO 기능 지연 때문에 온보딩이 막혔다.
```

추출 결과:

```
Entity:
- ACME 고객사
- SSO 기능
- 온보딩

Type:
- ACME 고객사: Customer
- SSO 기능: Product Feature
- 온보딩: Process
```

5.4 관계 추출

엔티티 사이의 관계를 뽑는다.

```
ACME 고객사 --blocked_by--> SSO 기능 지연
SSO 기능 --affects--> 온보딩
```

5.5 지식 그래프 생성

엔티티와 관계를 그래프 DB나 별도 저장소에 저장한다.

사용 가능한 저장 방식:

- Neo4j 같은 Graph DB
- RDF store
- NetworkX
- Vector DB와 Graph DB의 하이브리드 구조
- 자체 property graph 저장소

5.6 커뮤니티 탐지와 요약

그래프 안에서 서로 강하게 연결된 노드 묶음을 찾고, 이를 요약한다.

예:

```
Community 1: Enterprise Onboarding Issues
- ACME
- Globex
- SSO
- SCIM
- Security Review
- Onboarding Delay
```

요약:

> 여러 Enterprise 고객의 온보딩 지연은 SSO, SCIM, 보안 검토 프로세스와 관련되어 있으며, Platform Team의 병목이 반복적으로 등장한다.

이런 community summary는 전체 데이터셋을 요약하거나 패턴을 찾는 질문에 유용하다.

---

6. Query 방식

Graph RAG의 검색 방식은 크게 세 가지로 나눌 수 있다.

6.1 Local Search

특정 엔티티 중심 질문에 적합하다.

예:

> "ACME 고객사의 온보딩 이슈는 뭐야?"

흐름:

```
질문에서 ACME 식별
→ 그래프에서 ACME 노드 찾기
→ 연결된 이슈, 기능, 팀, 문서 chunk 탐색
→ 관련 context를 LLM에 제공
→ 답변 생성
```

Local Search는 특정 고객, 기능, 사람, 프로젝트처럼 명확한 대상이 있을 때 좋다.

6.2 Global Search

전체 데이터셋에 대한 요약이나 패턴 분석에 적합하다.

예:

> "지난 분기의 고객 피드백에서 가장 큰 리스크는 뭐야?"

흐름:

```
전체 그래프의 community summary 검색
→ 관련 community report 선택
→ 각 요약에서 부분 답변 생성
→ 최종 종합 답변 생성
```

Global Search는 "전체적으로 봤을 때 무엇이 중요한가?"를 묻는 질문에 강하다.

6.3 DRIFT Search

Local Search와 Global Search의 중간 형태다.

특정 엔티티 주변을 탐색하면서도, 관련 community summary를 함께 활용한다.

즉, 특정 대상에 대한 깊은 검색과 전체 맥락을 동시에 고려하는 방식이다.

---

7. 예시

다음과 같은 문서가 있다고 가정한다.

```
문서 1:
ACME는 SSO 미지원 때문에 계약 체결을 보류했다.

문서 2:
Globex도 SSO와 SCIM을 요구했으며, 보안 검토가 지연되고 있다.

문서 3:
Platform Team은 SSO보다 Billing API를 우선순위로 두고 있다.

문서 4:
Enterprise Sales 팀은 SSO 지연이 Q3 매출 리스크라고 보고했다.
```

일반 RAG는 “Q3 매출 리스크는?”이라는 질문에 문서 4만 가져와서 이렇게 답할 수 있다.

> Enterprise Sales 팀은 SSO 지연을 Q3 매출 리스크로 보고했습니다.

틀리지는 않지만 얕은 답변이다.

Graph RAG는 다음 관계를 함께 본다.

```
ACME --blocked_by--> SSO
Globex --requires--> SSO
Globex --requires--> SCIM
SSO --delayed_by--> Security Review
Platform Team --prioritizes--> Billing API
SSO Delay --risk_to--> Q3 Revenue
```

그러면 더 구조적인 답변이 가능하다.

> Q3 매출 리스크의 핵심은 Enterprise 고객 온보딩 지연입니다. ACME와 Globex 모두 SSO 또는 SCIM을 요구하고 있으며, SSO는 보안 검토와 Platform Team 우선순위 문제로 지연되고 있습니다. 따라서 단일 기능 문제가 아니라 Enterprise Sales, Platform Team, Security Review 간의 병목이 매출 리스크로 이어지는 구조입니다.

---

8. Graph RAG가 유용한 경우

Graph RAG는 다음 상황에 적합하다.

- 문서가 많고 서로 얽혀 있다.
- 사람, 조직, 프로젝트, 기능, 사건 사이의 관계가 중요하다.
- 단일 문서보다 여러 문서를 종합해야 한다.
- 전체 요약, 트렌드, 원인 분석 질문이 많다.
- 출처와 근거 추적이 중요하다.
- 일반 벡터 RAG가 관련 chunk를 놓치거나 엉뚱한 chunk를 가져온다.

예시 도메인:

- 사내 지식 검색
- 고객지원 분석
- 법률 문서 분석
- 연구 논문 분석
- 프로젝트 회고 및 의사결정 추적
- 보안 사고 분석
- 조직 내 dependency 분석

---

9. Graph RAG가 과한 경우

다음 경우에는 일반 RAG가 더 낫다.

- 문서 수가 적다.
- 질문이 단순 FAQ 위주다.
- 엔티티 간 관계가 중요하지 않다.
- 실시간 업데이트가 매우 중요하다.
- 그래프 구축과 검증 비용을 감당하기 어렵다.
- 데이터 품질이 낮아서 관계 추출 오류가 많다.

예를 들어 다음 질문은 일반 RAG로 충분한 경우가 많다.

> “환불 정책 알려줘.”
> “계약서 3조 요약해줘.”
> “이 문서의 핵심 내용만 정리해줘.”

---

10. 장점

10.1 관계 기반 추론이 강해짐

일반 RAG는 비슷한 문단을 찾지만, Graph RAG는 연결된 정보를 찾는다.

예:

```
고객 → 요청 기능 → blocker → 담당 팀 → 매출 리스크
```

이런 연결을 따라가며 답변할 수 있다.

10.2 전체 corpus 요약에 강함

Graph RAG는 community summary를 활용할 수 있기 때문에 전체 문서군에 대한 질문에 강하다.

예:

> “지난달 고객 피드백에서 반복적으로 등장한 문제는?”

10.3 출처 추적이 좋아질 수 있음

노드와 엣지를 원문 chunk에 연결해두면 답변의 근거를 추적하기 쉽다.

```
답변
→ 관련 엔티티
→ 관계
→ 원문 문서
```

다만 이는 그래프 구축과 citation 설계를 잘했을 때의 장점이다.

---

11. 단점과 어려움

11.1 구축 비용이 높음

일반 RAG:

```
chunk → embedding → vector DB
```

Graph RAG:

```
chunk
→ entity extraction
→ relationship extraction
→ entity resolution
→ graph construction
→ community detection
→ community summarization
→ graph retrieval
```

단계가 훨씬 많다.

11.2 엔티티 중복 문제

같은 대상을 여러 이름으로 부를 수 있다.

```
OpenAI
Open AI
오픈AI
OpenAI Inc.
```

이를 같은 엔티티로 합치는 과정이 필요하다. 이를 entity resolution 또는 deduplication이라고 한다.

11.3 관계 추출 오류

LLM이 관계를 잘못 뽑을 수 있다.

원문:

```
A 팀은 B 기능을 담당하지 않는다.
```

잘못된 추출:

```
A 팀 --owns--> B 기능
```

이런 오류가 생기면 Graph RAG가 오히려 잘못된 답변을 강화할 수 있다.

11.4 업데이트가 복잡함

문서가 바뀌면 그래프도 갱신해야 한다.

필요 작업:

- 새 문서 추가
- 기존 문서 수정 반영
- 삭제된 정보 제거
- 관계 재추출
- 커뮤니티 재계산
- 요약 재생성

---

12. 구현 아키텍처 예시

```
[Documents]
    ↓
[Chunking]
    ↓
[LLM Entity/Relation Extraction]
    ↓
[Knowledge Graph]
    ├─ Graph DB: Neo4j / RDF / NetworkX
    ├─ Vector DB: Qdrant / Weaviate / Pinecone / Chroma
    └─ Original Text Store
    ↓
[Retriever]
    ├─ Vector Search
    ├─ Graph Traversal
    ├─ Community Summary Search
    └─ Hybrid Ranking
    ↓
[LLM Answer Generation]
    ↓
[Answer + Citations]
```

실무에서는 Graph DB만 쓰기보다, **그래프 + 벡터 검색의 하이브리드 구조**를 많이 사용한다.

---

13. 검색 전략

13.1 Vector-first

먼저 벡터 검색으로 관련 chunk를 찾고, 그 chunk에 연결된 엔티티와 이웃 노드를 확장한다.

```
질문 → vector search → 관련 chunk → 연결된 노드 → 이웃 관계 확장
```

기존 RAG에 붙이기 쉬운 방식이다.

13.2 Entity-first

질문에서 엔티티를 먼저 뽑고, 그래프에서 해당 노드를 찾는다.

```
질문 → entity linking → graph node → neighborhood traversal
```

특정 고객, 프로젝트, 기능 중심 질문에 강하다.

13.3 Community-first

질문과 관련된 community summary를 먼저 찾는다.

```
질문 → community report 검색 → 관련 커뮤니티 요약 → 답변
```

전체 요약, 트렌드, 패턴 분석에 강하다.

13.4 Hybrid

실무에서는 보통 여러 방식을 섞는다.

```
vector search
+ entity search
+ graph traversal
+ community summaries
+ reranking
```

---

14. 간단한 의사코드

```
def graph_rag_answer(question):
    # 1. 질문에서 엔티티 추출
    query_entities = extract_entities(question)

    # 2. 벡터 검색으로 관련 chunk 검색
    relevant_chunks = vector_search(question)

    # 3. 엔티티를 그래프 노드에 매핑
    seed_nodes = link_entities_to_graph(query_entities)

    # 4. 그래프 주변 탐색
    subgraph = graph_traverse(seed_nodes, depth=2)

    # 5. 관련 커뮤니티 요약 검색
    community_reports = search_community_summaries(question, seed_nodes)

    # 6. context 구성
    context = build_context(
        chunks=relevant_chunks,
        subgraph=subgraph,
        summaries=community_reports
    )
```

```
    # 7. LLM 답변 생성
    return llm_generate(question, context)
```

---

15. 적용 예시

15.1 고객지원

질문:

> “최근 Enterprise 고객들이 가장 많이 겪는 문제는?”

Graph RAG는 고객, 기능, 티켓, 장애, 담당 팀, 계약 리스크를 연결해서 답할 수 있다.

15.2 법률 문서

질문:

> “이 계약서에서 책임 제한 조항과 해지 조항이 충돌하는 부분이 있어?”

조항, 당사자, 의무, 예외 조건, 참조 관계를 그래프로 만들면 유리하다.

15.3 연구 논문 분석

질문:

> “이 분야에서 서로 경쟁하는 방법론들은 어떻게 나뉘어?”

논문, 저자, 방법론, 데이터셋, 성능 지표를 그래프로 연결할 수 있다.

15.4 사내 지식 검색

질문:

> “이 프로젝트가 왜 지연됐는지 관련 회의록과 의사결정을 종합해줘.”

프로젝트, 담당자, 결정, blocker, 날짜, 팀 간 의존성을 연결할 수 있다.

---

16. 도입 전 체크리스트

Graph RAG를 도입하기 전에 다음 질문을 확인해야 한다.

- 문서 안에 사람, 조직, 제품, 사건, 개념이 자주 등장하는가?
- 이들 사이의 관계가 답변 품질에 중요한가?
- 단일 문서보다 여러 문서를 종합해야 하는가?

- 전체 요약, 트렌드, 원인 분석 질문이 많은가?
- 출처와 근거 추적이 중요한가?
- 일반 벡터 RAG가 자주 엉뚱한 chunk를 가져오는가?
- 그래프 구축과 검증 비용을 감당할 수 있는가?

---

17. 한 문장 요약

Graph RAG는 문서 속 엔티티와 관계를 그래프로 만들어, 단순한 문서 유사도 검색보다 관계·맥락·전체 구조를 더 잘 활용하는 RAG 방식이다.

쉽게 말하면:

> 일반 RAG는 관련 문단을 찾고,
> Graph RAG는 관련 문단뿐 아니라 그 문단 속 대상들이 어떻게 연결되어 있는지까지 찾아서 답한다.